



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ingeniería Industrial y de Sistemas

IIC2413 Base de Datos

Apuntes - Interrogación 2

Autor:

- Franco Nicolai Corvalán Mansourova

Fecha de edición: 25 de octubre de 2025

Índice de contenido

1 Clase 8: Lógica en la Base de Datos	3
1.1 Introducción:	3
1.2 Vistas (Views):	3
1.2.1 Vistas Normales (Tablas Virtuales)	3
1.2.2 B. Vistas Materializadas (Materialized Views)	4
1.3 Triggers (Gatillos):	6
1.4 Stored Procedures (Procedimientos Almacenados):	7
2 Clase 9: Schedules de Transacciones	10
2.1 Introducción:	10
2.2 Schedule (Cronograma)	11
2.2.1 A. Schedules Seriales (El objetivo)	11
2.2.2 B. Schedules Serializables (El método práctico)	11
2.3 Conflictos en la serializabilidad	12
2.3.1 Acciones Conflictivas	12
2.3.2 Conflicto Serializables	12
2.4 Control de Concurrencia con Locks	13
2.5 ACID y Niveles de Aislamiento	14
3 Clase 10: Transacciones y Recuperación de Fallas	16
3.1 Introducción:	16
3.2 La Arquitectura de Almacenamiento	16
3.3 El Log Manager	17
3.4 Estrategias de Logging	17
3.4.1 Undo Logging (Estrategia de Deshacer)	18
3.4.2 Redo Logging (Estrategia de Repetir)	18
3.4.3 Undo/Redo Logging (El Híbrido de Alto Rendimiento)	18
3.5 Checkpoints	19

4 Clase 11: Fundamentos de Estructuras de Datos	20
4.1 Introducción:	20
4.2 El Lienzo de la Memoria y los Punteros	20
4.3 Los Índices: La Solución al I/O	20
4.3.1 Tablas de Hash (<i>Hash Index</i>)	21
4.3.2 Árboles B+	21
4.4 Clustered vs. Unclustered: El Contenido de la Hoja	22
4.4.1 Índice Clustered (<i>Índice Primario</i>)	22
4.4.2 Índice Unclustered (<i>Índice Secundario</i>)	22

1. Clase 8: Lógica en la Base de Datos

1.1. Introducción:

En el desarrollo de software, la Lógica del Negocio (*Business Logic*) se refiere a todas las reglas y algoritmos que definen cómo debe funcionar el sistema (ejemplo: "Si un cliente es Premium, tiene un 10 % de descuento"). Tradicionalmente, gran parte de esta lógica se implementa en la capa de aplicación (código Python, Java, etc.). Sin embargo, los motores de bases de datos relacionales (como PostgreSQL o MySQL) ofrecen herramientas para implementar ciertas reglas directamente en la BDD. Esto se hace para:

- **Centralizar la Lógica:** Asegurar que las reglas se apliquen universalmente, sin importar desde dónde se acceda a la BDD.
- **Optimizar el Rendimiento:** Mover cálculos pesados al motor de la base de datos, que está optimizado para manipular grandes volúmenes de datos.
- **Garantizar la Consistencia de Datos (Integridad):** Asegurar que los datos sigan las reglas del negocio en todo momento.

1.2. Vistas (Views):

Coloquemos nos en el caso cuando tenemos una consulta muy compleja o muy frecuente que genera un resultado esencial como el promedio de notas de una película, una mala idea seria agregar una columna promedio en la tabla Películas, ya que si se agrega una nueva Evaluación, el valor precalculado en la tabla Películas queda obsoleto al instante, violando la integridad y la Tercera Forma Normal (3NF). dependiente de otros datos, no de la clave primaria). Es por ello que proponemos no guardar el dato, sino guardar la forma de calcularlo. Aquí es donde entran las Vistas.

1.2.1. Vistas Normales (Tablas Virtuales)

Una Vista Normal es una Tabla Virtual. No almacena datos por sí misma (no ocupa espacio en disco) sino que guarda la definición de una consulta SELECT.

Conceptos Clave de Vistas Normales

- **Definición:** Una consulta SELECT que se almacena en la BDD con un nombre, y que puede ser consultada como si fuera una tabla.
- **Sintaxis:** CREATE VIEW NombreVista AS SELECT ...

- **Mecanismo Clave:** Cuando se consulta la vista, el motor de la BDD reescribe internamente la consulta, reemplazando el nombre de la vista por la SELECT original, y luego ejecuta esa consulta compleja sobre las Tablas Base.
- **Actualización: Siempre actualizados (en tiempo real).** Dado que la consulta se ejecuta siempre sobre los datos originales en el momento del acceso, cualquier cambio en las tablas base se refleja inmediatamente.
- **Ventajas:**
 - **Abstracción:** Oculta la complejidad de múltiples JOINS o cálculos al usuario.
 - **Mantenibilidad:** Si la lógica del cálculo cambia (ej: el promedio ahora excluye la nota más baja), solo se modifica la vista en un solo lugar.
- **Desventaja:**
 - **Rendimiento:** Es inherentemente **más lenta** que una tabla real, ya que la costosa operación (AVG(), GROUP BY, JOIN) se ejecuta cada vez que se consulta la vista.

El caso previamente planteado se puede resolver mediante el siguiente ejemplo de implementación:

```
1 -- 1. Creación de la Vista Normal
2 CREATE VIEW PelEval AS
3 SELECT pelicula, director,
4        AVG(eval) AS promedio
5 FROM Evaluaciones
6 GROUP BY pelicula, director;
7
8 -- 2. Consulta simplificada a la Vista
9 SELECT promedio
10 FROM PelEval
11 WHERE pelicula = 'The Prestige' AND director = 'Christopher Nolan';
```

Imagen 1

Código ejemplo de los contenidos explicados anteriormente

1.2.2. B. Vistas Materializadas (Materialized Views)

Las Vistas Normales resuelven la consistencia, pero introducen una latencia en el rendimiento. ¿Qué pasa si la consulta es muy pesada, pero no necesitamos datos actualizados al segundo? Pues usaremos las vistas Materializadas.

Conceptos Clave de Vistas Materializadas

- **Definición:** Una Vista que **SÍ crea y mantiene una Tabla Física en el disco**. Almacena el resultado de la consulta como una caché.
- **Sintaxis:** `CREATE MATERIALIZED VIEW NombreVista AS SELECT ...`
- **Mecanismo Clave: Almacenamiento:** Cuando consultas la vista materializada, la BDD lee directamente los resultados pre-calculados del disco, como si fuera una tabla normal. No se reescribe ni se ejecuta la consulta pesada.
- **Actualización: NO se actualizan automáticamente.** El resultado se vuelve obsoleto inmediatamente después de un cambio en las tablas base.
- **Comando de Actualización:** Requieren un comando explícito para recalcular el resultado: `REFRESH MATERIALIZED VIEW NombreVista;`
- **Ventajas: Rendimiento Máximo:** La consulta es extremadamente rápida (velocidad de lectura de tabla) al evitar cálculos complejos en el momento de la consulta.
- **Desventajas:**
 - **Obsolescencia:** Los datos no están actualizados en tiempo real.
 - **Redundancia / Almacenamiento:** Requiere espacio en disco y se debe gestionar la consistencia.
- **Uso Ideal: Dashboards,** reportes analíticos o datos históricos donde la velocidad es crucial y una latencia de minutos u horas es aceptable (ej: promedio de ventas del día anterior).

El caso previamente planteado se puede resolver mediante el siguiente ejemplo de implementación:

```
1 -- 1. Creación de la Vista Materializada
2 CREATE MATERIALIZED VIEW PelEval_MV AS
3 SELECT
4     pelicula,
5     director,
6     AVG(eval) AS promedio
7 FROM
8     Evaluaciones
9 GROUP BY
10    pelicula,
11    director
12 WITH DATA;
13
14 -- 2. Consulta simplificada a la Vista Materializada
15 SELECT
16     promedio
17 FROM
18     PelEval_MV
19 WHERE
20     pelicula = 'The Prestige' AND director = 'Christopher Nolan';
21
22 -- 3. Actualización de la Vista Materializada (Clave de la diferencia)
23 REFRESH MATERIALIZED VIEW CONCURRENTLY PelEval_MV;
```

Imagen 2

Código ejemplo de los contenidos explicados anteriormente

1.3. Triggers (Gatillos):

Ya sabemos que la Vista Materializada es rápida, pero requiere que alguien ejecute REFRESH manualmente para mantener la consistencia. Aquí es donde los Triggers nos ayudan a automatizar esta tarea. Un Trigger es un Procedimiento (una pieza de código) que se define en el DBMS y se "gatilla" o ejecuta automáticamente cada vez que ocurre un evento específico sobre una tabla.

Características de los Triggers

- **Función:** Forzar reglas de negocio complejas y mantener la consistencia e integridad de la BDD de forma **reactiva**, o sea, reacciona ante eventos.
- **Eventos (Ejemplos):** Se activan ante operaciones como INSERT, UPDATE, o DELETE (en las tablas base).
- **Momento de Ejecución:**

- **BEFORE:** Se ejecuta antes del evento (ideal para validación de datos).
- **AFTER:** Se ejecuta después del evento (ideal para acciones secundarias o auditoría).
- **INSTEAD OF:** Se ejecuta en lugar del evento (usado para actualizar vistas no modificables directamente).

■ **Alcance:**

- **FOR EACH ROW:** El disparador se ejecuta una vez por cada fila afectada por la sentencia SQL.
- **FOR EACH STATEMENT:** El disparador se ejecuta solo una vez por la sentencia SQL completa, sin importar cuántas filas afecte.

Dado el ejemplo usado anteriormente, buscaremos la automatizando la actualización de la vista materializada. Para resolver el problema de la obsolescencia de PelEval, podemos crear un Trigger que se ejecute después de cada inserción en la tabla base:

```
1 CREATE TRIGGER refresh_eval
2 AFTER INSERT ON Evaluaciones
3 FOR EACH STATEMENT
4 BEGIN
5     REFRESH MATERIALIZED VIEW PelEval;
6 END;
```

Imagen 3

Código ejemplo de los contenidos explicados anteriormente

1.4. Stored Procedures (Procedimientos Almacenados):

Finalmente, necesitamos una herramienta para ejecutar tareas complejas que no están atadas a un evento específico, sino que el programador desea llamar bajo demanda. Aquí es donde entra la programación procedural. Los Procedimientos Almacenados son bloques de código (funciones definidas en SQL) que quedan guardados en el DBMS para ser ejecutados repetidamente desde la aplicación.

Características de los Procedimientos Almacenados

- **Lenguaje:** Se usa un lenguaje **procedural** (ej: PL/pgSQL en PostgreSQL, T-SQL en SQL Server) que añade estructuras de control (IF/THEN/ELSE, FOR LOOP) al SQL estándar.

- **Función:** Ejecutar lógica compleja y repetitiva directamente en el servidor. En un solo proceso, pueden ejecutar múltiples SELECT, INSERT, UPDATE o DELETE (control de flujo).
- **Sintaxis Base:** CREATE OR REPLACE FUNCTION nombre_funcion (argumentos) RETURNS tipo_retorno AS \$\$... \$\$ LANGUAGE plpgsql
- **Ventaja Principal: Velocidad/Rendimiento:** Evitan la latencia de red al ejecutar múltiples operaciones cerca de los datos. Son altamente optimizados.
- **Desventaja Principal: Portabilidad:** El código procedural es altamente dependiente del motor de la base de datos, haciendo casi imposible migrar a otro DBMS sin reescribir la lógica.

Componentes de PL/pgSQL

- **DECLARE:** Se utiliza para definir variables locales dentro de la función.
- **BEGIN ... END:** Contiene el cuerpo principal de la lógica, que incluye sentencias SQL y estructuras de control.
- **IF / ELSE IF / ELSE:** Permite el control de flujo condicional.
- **FOR variable IN iterable LOOP ... END LOOP:** Permite iterar sobre el resultado de una consulta SELECT o un rango.
- **RETURN:** Devuelve un valor o el resultado de una consulta (RETURNS TABLE para consultas dinámicas).

Apliquemos lo anterior con un ejemplo, un procedimiento para transferir la información de una tabla a otra (Concatenar nombre y apellido):

```
1 CREATE OR REPLACE FUNCTION transferencia_nombres() RETURNS void AS $$
2 DECLARE
3     tupla RECORD;
4     concat_nombre varchar;
5 BEGIN
6     FOR tupla IN SELECT * FROM Personas LOOP
7         concat_nombre = tupla.nombre || ' ' || tupla.apellido;
8         INSERT INTO PersonasCompleto VALUES (tupla.rut, concat_nombre);
9     END LOOP;
10    RETURN;
11 END
12 $$ language plpgsql;
13
14 SELECT transferencia_nombres();
```

Imagen 4

Código ejemplo de los contenidos explicados anteriormente

2. Clase 9: Schedules de Transacciones

2.1. Introducción:

En un sistema de Base de Datos moderno, miles de usuarios ejecutan operaciones **simultáneamente** (*conurrencia*). Para que el sistema sea eficiente, el DBMS debe intercalar estas operaciones. El desafío es: ¿Cómo mezclamos las operaciones para ser rápidos sin corromper los datos?

El Problema Fundamental es la inconsistencia por intercalación dos transacciones concurrentes (T_1 y T_2) intentan modificar el mismo dato.

Definición: Una transacción es una secuencia de 1 o más operaciones que modifican o consultan la base de datos (Transferencias de dinero entre cuentas, compra por internet, inscribir un curso).

Sea el siguiente ejemplo:

Transacción (T1)	Transacción (T2)	Saldo Cuenta (A)
T1: $R(A)$ (Lee 1000)		1000
T1: $W(A)$ (Escribe 900)		900
	T2: $R(A)$ (Lee 900)	900
	T2: $W(A)$ (Escribe 700)	700
T1: $R(B)$ (Lee 1000)		700
T1: $W(B)$ (Escribe 1100)		700
	T2: $R(B)$ (Lee 1000)	700
	T2: $W(B)$ (Escribe 1200)	700
¡Error grave!		

Imagen 5

Ejemplo de Intercalación Insegura con Pérdida de Actualización.

El concepto de pérdida de actualización se refiere a una anomalía de concurrencia en bases de datos donde la ejecución no controlada de múltiples transacciones lleva a un estado incorrecto. Este problema ocurre cuando una transacción (T_1) lee un valor de dato que otra transacción concurrente (T_2) ya ha modificado y escrito en la base de datos, para luego sobrescribir esa modificación de T_2 con un valor basado en su propia lectura obsoleta. En el ejemplo mencionado, la transacción T_1 lee inicialmente un valor obsoleto (1000) y, tras realizar un cálculo, escribe 1100, invalidando la

actualización de T_2 (que había escrito 1200). El problema central es que esta intercalación incontrolada de operaciones de lectura y escritura ($R(B) \rightarrow W(B)$) viola la consistencia e integridad de los datos, dando como resultado un valor final que no refleja todas las operaciones realizadas por las transacciones.

2.2. Schedule (Cronograma)

Para que el DBMS pueda controlar el caos de la concurrencia, necesita un plan. Este plan se llama Schedule.

Conceptos Clave de Schedules

- **Definición de Schedule (S):** Una secuencia de operaciones primitivas (R, W, A, C) de una o más transacciones. Es la partitura que define el orden de ejecución.
- **Restricción Clave:** El DBMS puede mezclar operaciones de diferentes transacciones, pero **NO** puede alterar el orden de las operaciones dentro de una misma transacción.
- **Operaciones Primitivas:**
 - $R_i(X)$: Transacción i lee el dato X .
 - $W_i(X)$: Transacción i escribe el dato X .
 - A : Abort (Fallar).
 - C : Commit (Confirmar).

2.2.1. A. Schedules Seriales (El objetivo)

Un schedule es Serial si no hay ninguna intercalación, corresponde al plan más seguro. Es relevante aclarar que una transacción se completa enteramente (con Commit o Abort) antes de que la siguiente comience.

- **Ventaja:** Garantizan la **Consistencia** absoluta. El resultado siempre es predecible y correcto.
- **Desventaja:** Son **Ineficientes** y Lentos (no aprovechan la concurrencia del hardware).

2.2.2. B. Schedules Serializables (El método práctico)

El objetivo del *Transaction Manager* del DBMS es lograr un schedule que sea tan rápido como la ejecución intercalada, pero tan seguro como la ejecución serial.

- **Definición:** Un schedule S es **Serializable** si es equivalente a algún schedule serial S' con las mismas transacciones.

- **Significado:** Si la ejecución mezclada S produce el mismo estado final en la base de datos que una ejecución secuencial S' , entonces S es seguro.

2.3. Conflictos en la serializabilidad

¿Cómo sabe el DBMS si un schedule es serializable sin ejecutarlo? Analizando los conflictos.

2.3.1. Acciones Conflictivas

Dos acciones ($Op_i(X)$ y $Op_j(X)$) de dos transacciones diferentes (T_i y T_j) sobre el mismo recurso (X) entran en conflicto si y solo si **al menos una de ellas es una operación de escritura (W)**.

Tipos de Conflictos en Schedules

- **Lectura-Escritura (RW)**
 - **Ejemplo:** $R_i(X), W_j(X)$
 - **Permutable:** NO
 - **Problema Asociado:** Lectura No Repetible (*Unrepeatable Read*)
- **Escritura-Lectura (WR)**
 - **Ejemplo:** $W_i(X), R_j(X)$
 - **Permutable:** NO
 - **Problema Asociado:** Lectura Sucia (*Dirty Read*)
- **Escritura-Escritura (WW)**
 - **Ejemplo:** $W_i(X), W_j(X)$
 - **Permutable:** NO
 - **Problema Asociado:** Actualización Perdida (*Lost Update*)
- **No Conflictivas**
 - **Ejemplo:** $R_i(X), R_j(X)$ o $Op_i(X), Op_j(Y)$
 - **Permutable:** SÍ
 - **Problema Asociado:** N/A

2.3.2. Conflicto Serializables

Un schedule es **Conflict Serializable** si podemos transformarlo en un schedule serial equivalente usando únicamente permutaciones de acciones **no conflictivas**.

- **Teorema:** Un schedule es Conflict Serializable si y solo si su **Grafo de Precedencia** es acíclico.

Grafo de Precedencia

El Grafo de Precedencia es la herramienta que usa el DBMS para verificar la serializabilidad de forma eficiente.

- **Nodos:** Representan a cada una de las transacciones ($T_1, T_2, T_3, \dots$).
- **Aristas:** Se dibuja una arista dirigida de $T_i \rightarrow T_j$ si T_i realiza una operación conflictiva antes que T_j sobre el mismo dato.
- **Regla de Consistencia:** Si el grafo tiene un **Ciclo**, significa que existe una dependencia mutua en conflicto (ej: $T_1 \rightarrow T_2$ y $T_2 \rightarrow T_1$), por lo tanto, el schedule **NO** es conflict serializable y es inseguro.

2.4. Control de Concurrency con Locks

El *Transaction Manager* utiliza protocolos para asegurar que solo se permitan schedules serializables. El más común es el **Protocolo de Bloqueo en Dos Fases Estricto** (*Strict Two-Phase Locking - Strict 2PL*). En palabras simples, las transacciones piden bloqueos (*locks*) sobre los recursos que necesitan antes de usarlos.

Tipos de Locks y Compatibilidad

- **Shared Lock (*S-Lock*):**
 - Se pide para leer ($R(X)$).
 - **Regla:** Compatible con otros S-Locks.
- **Exclusive Lock (*X-Lock*):**
 - Se pide para escribir ($W(X)$).
 - **Regla:** Incompatible con cualquier otro lock (S o X). Solo una transacción a la vez.

Reglas Clave de Strict 2PL

- **Fase de Adquisición (*Lock Phase*):** Una transacción debe adquirir **todos** los locks que necesita (S o X) antes de comenzar cualquier operación.
- **Fase de Liberación (*Unlock/Release Phase*):** Una transacción solo libera **todos** sus locks **al final**, después de hacer COMMIT o ABORT. (Esto es lo que lo hace *Strict*).

Problemas Asociados a Locks

- **Deadlocks (Interbloqueos):** Ocurren cuando dos o más transacciones esperan indefinidamente por un lock que tiene la otra, creando un ciclo de espera.
- **Solución:** El DBMS usa un Detector de Deadlocks que identifica el ciclo y **aborta** (*ROLLBACK* a) una de las transacciones para liberar los recursos.
- **Long Transactions:** Transacciones que tardan mucho tiempo en completarse (ej: análisis financiero de fin de mes) y que retienen locks por periodos prolongados, degradando el rendimiento de las demás transacciones.

2.5. ACID y Niveles de Aislamiento

Las transacciones buscan asegurar las propiedades **ACID** (*Atomicidad, Consistencia, Aislamiento y Durabilidad*). El DBMS permite diferentes **Niveles de Aislamiento en SQL** para permitir más velocidad a costa de una menor seguridad:

Niveles de Aislamiento en SQL y Anomalías Permitidas

- **SERIALIZABLE**
 - **Garantiza Serializabilidad:** SÍ
 - **Permite Dirty Read:** NO
 - **Permite Unrepeatable Read:** NO
 - **Permite Phantom Read:** NO
- **REPEATABLE READ**
 - **Garantiza Serializabilidad:** NO
 - **Permite Dirty Read:** NO
 - **Permite Unrepeatable Read:** NO
 - **Permite Phantom Read:** SÍ
- **READ COMMITTED**
 - **Garantiza Serializabilidad:** NO
 - **Permite Dirty Read:** NO
 - **Permite Unrepeatable Read:** SÍ
 - **Permite Phantom Read:** SÍ

■ READ UNCOMMITTED

- **Garantiza Serializabilidad:** NO
- **Permite Dirty Read:** SÍ
- **Permite Unrepeatable Read:** SÍ
- **Permite Phantom Read:** SÍ

Fallas de Aislamiento (Anomalías)

- **Dirty Read** (*Lectura Sucia*): Leer un dato que fue escrito por otra transacción que aún no ha hecho COMMIT (y podría hacer ABORT).
 - Nivel Mínimo para Evitar: **READ COMMITTED**.
- **Unrepeatable Read** (*Lectura No Repetible*): Una transacción lee un dato dos veces y obtiene un valor diferente entre lecturas porque otra transacción lo modificó y confirmó (COMMIT) en el medio.
 - Nivel Mínimo para Evitar: **REPEATABLE READ**.
- **Phantom Read** (*Lectura Fantasma*): Una transacción realiza una consulta de rango dos veces y encuentra nuevas filas (o menos filas) en la segunda lectura, añadidas por otra transacción confirmada.
 - Nivel Mínimo para Evitar: **SERIALIZABLE**.

3. Clase 10: Transacciones y Recuperación de Fallas

3.1. Introducción:

Nuestra historia de la confiabilidad comienza con la **Transacción**. Piensa en ella como una secuencia de operaciones (como restar dinero de una cuenta y sumarlo a otra) que el Sistema de Gestión de Bases de Datos (DBMS) trata como una única unidad indivisible de trabajo. El propósito de la transacción es ser la garantía de que, si algo sale mal, el sistema mantendrá su integridad.

Esta garantía se materializa en los cuatro pilares: las **propiedades ACID**.

La Narrativa de las Propiedades ACID

- **A - Atomicidad** (*"Todo o Nada"*): Esta es la regla fundamental. O se ejecutan todas las operaciones que componen la transacción, o ninguna de ellas tiene efecto. Si, por ejemplo, se corta la energía a mitad de una transferencia bancaria, la Atomicidad asegura que el sistema sepa deshacer la parte que se hizo, devolviendo el dinero a su estado original para que no desaparezca.
- **C - Consistencia**: La Consistencia garantiza que la transacción moverá la base de datos de un estado válido a otro válido. Si la base de datos tiene una regla de negocio (como "el saldo de una cuenta nunca puede ser negativo"), ninguna transacción puede violar esa regla. Si una operación intenta romper una restricción de integridad, la Consistencia detiene la transacción.
- **I - Aislamiento** (*Isolation*): En un mundo de múltiples usuarios, el Aislamiento asegura que cada transacción se ejecute como si estuviera sola. Las transacciones concurrentes no deben interferir entre sí. Esto se resuelve a través de Schedules y Locks, y es gestionado por el *Transaction Manager*.
- **D - Durabilidad**: Una vez que una transacción ha sido confirmada (COMMIT), la Durabilidad es la promesa final: sus cambios son permanentes e irrevocables, incluso si el sistema entero falla, hay un corte de energía o el servidor se reinicia.

El **Log and Recovery Manager** es el componente clave encargado de asegurar la Atomicidad y la Durabilidad.

3.2. La Arquitectura de Almacenamiento

Para cumplir con la Durabilidad, el DBMS debe ser consciente de dónde y cómo se guardan los datos.

Jerarquía de Almacenamiento

- En la cima está la **RAM** (Memoria Principal), que es rápida pero **volátil** (los datos se pierden al fallar).
- En la base está el **Disco** (SSD/HDD), que es lento pero **no volátil** (los datos son permanentes).

El DBMS usa el **Buffer Pool**, un área reservada en la RAM, como un puente. Las tuplas que se necesitan manipular se cargan desde el disco en paquetes llamados **Páginas** o **Bloques**.

El cuello de botella que limita el rendimiento es el **Costo de I/O** (*Input/Output*) al acceder al disco. El tiempo más lento de ese acceso es el *Tiempo de Búsqueda* (*Seek Time*) del cabezal en el disco duro (HDD). Por eso, el DBMS intenta minimizar los viajes al disco a toda costa, manteniendo las páginas en el Buffer Pool. Para la fiabilidad del almacenamiento, se utiliza **RAID** (*Matriz Redundante de Discos*), que aumenta tanto el rendimiento como la capacidad de recuperación ante fallos físicos de disco.

Técnicas Principales de un RAID:

- Segmentación de Datos (Data Striping): Los datos se dividen en bloques (unidades de segmentación) distribuidos en varios discos.
- Redundancia: Uso de discos adicionales para permitir la reconstrucción de datos en caso de fallos.

3.3. El Log Manager

La Atomicidad y la Durabilidad solo se pueden garantizar con un registro histórico impecable: el **Log**. El *Log Manager* escribe un diario secuencial en disco, documentando absolutamente todas las acciones. El sistema se apoya en el principio **Write-Ahead Logging (WAL)**, que define reglas estrictas sobre qué se escribe primero: ¿el Log o los Datos?

Registros Esenciales del Log

- <START T>: El inicio de una transacción.
- <COMMIT T>: La confirmación exitosa de una transacción.
- <ABORT T>: La cancelación de una transacción.
- < $T, X, V_{antiguo}, V_{nuevo}$ >: El registro más importante, que detalla que la transacción T modificó el dato X , y guarda tanto el valor antiguo como el nuevo.

3.4. Estrategias de Logging

La decisión de qué se escribe primero (el Log o los Datos) define la estrategia de Logging, y afecta directamente a la velocidad de COMMIT y la complejidad del *Recovery*.

3.4.1. Undo Logging (Estrategia de Deshacer)

El enfoque aquí es la seguridad de la Atomicidad (la "A").

- **Regla Clave:** Para hacer COMMIT, primero se deben forzar los **Datos** al disco y luego se escribe el Log de COMMIT.
- **Log de Cambio:** Solo necesita guardar el **Valor Antiguo** ($V_{antiguo}$) porque el objetivo del *Recovery* es deshacer cambios (**UNDO**).
- **Consecuencia:** El COMMIT es **lento** porque la transacción debe esperar a que todas las páginas modificadas lleguen al disco. Esto crea congestión de escritura.
- **Recovery:** Se lee el Log de atrás hacia adelante. Si una transacción no hizo COMMIT, se le aplica **UNDO** a sus cambios (se le devuelve el $V_{antiguo}$).

3.4.2. Redo Logging (Estrategia de Repetir)

El enfoque aquí es la velocidad y la Durabilidad (la "D").

- **Regla Clave:** Para hacer COMMIT, primero se escribe el **Log** de COMMIT al disco y luego, en algún momento posterior, se escriben los Datos.
- **Log de Cambio:** Solo necesita guardar el **Valor Nuevo** (V_{nuevo}) porque el objetivo del *Recovery* es repetir cambios (**REDO**).
- **Consecuencia:** El COMMIT es **rápido** porque la transacción solo necesita forzar un pequeño Log, sin esperar a los datos.
- **Recovery:** Se lee el Log de adelante hacia atrás. Si una transacción tiene COMMIT pero no tiene $< ENDT >$ (lo que significa que los datos pueden no haber llegado a disco), se le aplica **REDO** a sus cambios (se le vuelve a escribir el V_{nuevo}).

3.4.3. Undo/Redo Logging (El Híbrido de Alto Rendimiento)

Esta es la solución de mayor rendimiento, mezclando ambas para optimizar el COMMIT y la recuperación.

- **Log de Cambio:** Guarda ambos valores ($V_{antiguo}$ y V_{nuevo}).
- **Regla Clave:** El Log de COMMIT puede ir al disco en un momento arbitrario (antes o después de los datos). Esto permite el COMMIT más rápido.
- **Recovery:** Se realiza una recuperación dual:

- **UNDO** de las transacciones incompletas (sin COMMIT) usando $V_{antiguo}$.
- **REDO** de las transacciones confirmadas (con COMMIT pero sin $\langle ENDT \rangle$) usando V_{nuevo} .

3.5. Checkpoints

El proceso de *Recovery* es lento si hay que leer todo el Log. Los **Checkpoints** son puntos de referencia que limitan el trabajo de la recuperación.

- **Non-Quiescent Checkpoints:** No requieren detener la base de datos. El DBMS escribe un registro especial $\langle STARTCKPT(T_1, \dots, T_n) \rangle$ listando las transacciones activas. Cuando todas terminan, escribe $\langle ENDCKPT \rangle$.
- **Propósito:** Tras un fallo, el *Recovery Manager* solo necesita hacer **UNDO/REDO** desde la transacción activa más antigua que se listó en ese *Checkpoint*, en lugar de leer todo el histórico del Log.

En esencia, la recuperación de fallas es un proceso meticuloso de mirar el diario de a bordo (el Log) para asegurar que la promesa **ACID** se cumpla, sin importar lo que haya fallado.

4. Clase 11: Fundamentos de Estructuras de Datos

4.1. Introducción:

En un Sistema de Gestión de Bases de Datos (DBMS), el enemigo principal de la velocidad es el **Costo de I/O** (*Input/Output*), es decir, el tiempo que tarda en transferir datos desde el lento Disco Duro (Memoria Secundaria) a la rápida RAM (Memoria Principal). Para ello debemos tener en consideración lo siguiente:

- Para el DBMS, el costo de una consulta se mide en el **número de páginas** (bloques de disco) que deben leerse.
- Sin un índice, cualquier consulta debe realizar un **escaneo de tabla completo** (*full table scan*), leyendo todas las páginas.
- El costo de esta lectura secuencial es $\lceil \frac{n}{B} \rceil$, donde n es el número de tuplas y B es la capacidad de tuplas por página. Dicha relación ocupa $\lceil \frac{|R|}{B} \rceil$.

En palabras simples, para bases de datos grandes, este costo es insostenible. Necesitamos estructuras que permitan saltar directamente a la página que contiene el dato.

4.2. El Lienzo de la Memoria y los Punteros

Antes de construir índices, debemos entender los materiales básicos de la computación:

- **Memoria RAM:** Se imagina como una gran tabla de bits, donde cada fila tiene una dirección única.
- **Puntero:** Es una variable cuyo valor es la **dirección de memoria** de otra variable. Son el mapa que permite crear conexiones lógicas entre datos que no están contiguos.

Estructuras de Datos Fundamentales (EDD)

- **Arreglos:** Secuencias de celdas con largo fijo y se almacenan en memoria de manera **contigua**. Su fuerza es la **velocidad de acceso por posición**.
- **Listas Ligadas:** Secuencias de celdas con largo variable que se almacenan de manera aleatoria, pero se conectan entre sí mediante **punteros**. Su fuerza es la **flexibilidad dinámica**.

4.3. Los Índices: La Solución al I/O

Los **Índices** son estructuras de datos auxiliares diseñadas para optimizar el acceso a los datos, permitiendo encontrar tuplas sin necesidad de leer la tabla completa.

Definiciones Clave

- **Search Key:** Es el atributo que se utiliza para buscar algo en el índice.
- **Data Entry:** Es el resultado que retorna el índice después de buscar el *Search Key* (la tupla misma o un puntero a la tupla).

Con esto mencionado, existen métodos que usan índices, que son los siguientes:

4.3.1. Tablas de Hash (*Hash Index*)

- **Mecanismo:** Una función de hash ($h(k)$) mapea el *Search Key* (k) a un **bucket** primario, que es una página de disco.
- **Colisiones:** Se resuelven con **encadenamiento** a páginas de desbordamiento (*overflow pages*), formando una lista ligada de páginas.
- **Ventaja:** Ideal para **Consultas de Igualdad** (ej: WHERE aid = 12). El costo ideal es de **1 I/O**.
- **Desventaja Crítica:** Es muy ineficiente para **Consultas de Rango** (ej: WHERE aid > 12), ya que el hashing dispersa los valores de forma aleatoria, obligando a un escaneo completo de la tabla (Peor Caso: $O(\lceil \frac{|R|}{B} \rceil)$).

4.3.2. Árboles B+

Es la estructura de índice **más importante** y utilizada, ya que ofrece la mejor solución para búsquedas de rango.

- **Estructura:** Es un **árbol balanceado** donde cada nodo es una página de disco. La altura (h) es logarítmica ($O(\log N)$).
- **Nodos Internos (Directorios):** Contienen llaves y punteros para guiar la búsqueda.
- **Nodos Hoja (Datos):** Contienen los *Data Entry* y están **ordenadas** por el *Search Key*.
- **Encadenamiento de Hojas:** Las hojas están unidas con punteros para un **recorrido secuencial** eficiente.
- **Búsqueda Exacta:** Costo de $O(h)$ I/Os (se navega desde la raíz hasta la hoja).
- **Búsqueda de Rango:**
 1. Se navega a la hoja de inicio ($O(h)$ I/Os).
 2. Se utiliza el **encadenamiento de hojas** para recorrer horizontalmente las páginas en orden. Esto es extremadamente eficiente.

4.4. Clustered vs. Unclustered: El Contenido de la Hoja

La diferencia radica en qué guarda la hoja del índice (el *Data Entry*).

4.4.1. Índice Clustered (*Índice Primario*)

- **Data Entry:** La hoja del índice contiene la **tupla de datos completa**.
- **Mecanismo:** La tabla está **físicamente ordenada** en el disco según la llave del índice.
- **Limitación:** Solo puede haber **uno** por tabla.
- **Costo Búsqueda Exacta (B+ Tree):** Solo h I/Os (la tupla se obtiene directamente).
- **Costo Búsqueda de Rango (B+ Tree):** $h + B_m$ I/Os (es el caso más eficiente, ya que las páginas se leen secuencialmente).

4.4.2. Índice Unclustered (*Índice Secundario*)

- **Data Entry:** La hoja del índice contiene un **puntero** a la tupla real de datos.
- **Mecanismo:** El índice es un archivo separado y la tabla de datos **no tiene un orden físico** relacionado con esta llave.
- **Ventaja:** Se pueden crear **múltiples** índices *Unclustered* por tabla.
- **Costo Búsqueda Exacta (B+ Tree):** $h + 1$ I/Os (h para el puntero, más 1 I/O adicional para seguir el puntero).
- **Costo Búsqueda de Rango (B+ Tree):** **Peor caso.** Se puede requerir una I/O separada por cada tupla que calce con el rango, ya que están dispersas aleatoriamente en el disco.